

REPORT

DESIGN AND ANALYSIS OF A SNAKE GAME

Using C++ with OOP and Data Structures

COURSE

Object-Oriented Programming | Data Structures | Programming Fundamentals

TEAM MEMBERS

Ibrar Ahmed	M. Sanan	Hanzala Baqir
Implementation	Documentation	Testing & Analysis

1. Project Overview & Objective

Objective

To design, implement, and analyze a Snake game using C++, applying concepts of Object-Oriented Programming (OOP), Data Structures (DS), and Programming Fundamentals (PF). The project focuses on understanding algorithm behavior through movement logic, collision detection, and dynamic memory handling.

Problem Statement

The project simulates a classic Snake game where the user controls the snake using keyboard inputs (W, A, S, D). The snake moves on a 2D grid, grows when it eats food, and the game ends when it collides with walls or itself.

Technology Stack

Language	C++ (Standard C++11 and above)
Paradigm	Object-Oriented Programming (OOP)
Data Structure	Point body[] — fixed-size array with shift-based movement
Platform	Windows / Linux / macOS (cross-platform)
Controls	W = Up, A = Left, S = Down, D = Right, Q = Quit
Input Method	ENTER to start; WASD keys during gameplay

2. Features & Gameplay

Feature	Description
User-Controlled Snake	W/A/S/D keys control UP/LEFT/DOWN/RIGHT movement in real-time
Enter to Start	Welcome screen waits for ENTER keypress before game begins
Dynamic Snake Growth	Snake body lengthens by one segment each time food is consumed
Random Food Generation	Food spawns at a random valid cell, never overlapping the snake body
Score Display	Live score shown on HUD during gameplay (+10 points per food)
High Score Tracking	Best score saved to highscore.txt and persisted across sessions
Collision Detection	Game ends on wall collision $O(1)$ or self-collision $O(n)$

Play Again Prompt	After game over, player can restart or exit with Y/N input
180-Degree Reversal Block	Prevents snake from instantly reversing into itself
Cross-Platform Support	Runs on Windows, Linux, and macOS with platform-specific input

3. OOP Design & Program Structure

The program applies three core OOP principles: Encapsulation, Abstraction, and Composition. Each class has a single responsibility and hides its internal data.

3.1 Class: Point (Struct)

Represents a single (x, y) coordinate on the 2D grid. Used as the fundamental unit stored inside the snake body vector and for food position.

```
struct Point {
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const Point& o) const {
        return x == o.x && y == o.y;
    }
};
```

3.2 Class: Food

Encapsulates the food's position and random respawn logic. The private position variable is only accessible via the getPosition() getter.

- Private: Point position
- Public: Food() — constructor, calls respawn()
- Public: void respawn(occupied) — places food at random valid cell (O(n))
- Public: Point getPosition() — getter

3.3 Class: ScoreManager

Handles current game score, all-time high score, and file persistence. Score is private and can only be modified through class methods.

- Private: int score, int highScore
- Private: void loadHighScore() — reads from highscore.txt
- Private: void saveHighScore() — writes to highscore.txt
- Public: void addPoints(pts) — adds score, updates high score
- Public: void reset() — resets current score to 0

3.4 Class: Snake

Core data structure class. Uses a fixed-size Point array (body[MAX_LEN]) with shift-based movement: all segments shift one slot right each tick, the new head is placed at index 0, and the tail slot is simply overwritten when sliding.

- Private: Point body[MAX_LEN] — fixed-size array storing all body segments
- Private: Direction dir — current movement direction
- Public: void reset() — initialise snake at centre, length 3
- Public: void move(grew) — Dynamic Body Update Algorithm $O(n)$
- Public: bool hitWall() — Wall Collision Detection $O(1)$
- Public: bool hitSelf() — Self Collision Detection $O(n)$
- Public: bool ateFood(food) — food overlap check $O(1)$
- Public: void setDirection(d) — update direction, block 180-reversal

3.5 Class: SnakeGame (Main Controller)

Aggregates Snake, Food, and ScoreManager through composition. Implements the classic game loop pattern.

```
class SnakeGame {
private:
    Snake      snake;
    Food       food;
    ScoreManager scores;
    bool       gameOver;
    bool       quit;
    char       grid[HEIGHT][WIDTH + 1];
public:
    void setup();    // initialise round
    void draw();     // render to console
    void input();    // non-blocking keyboard
    void logic();    // game tick / state update
    void run();      // main game loop
};
```

3 (Updated). Multi-Platform Visualization Layer

3.2 Web-Based Visualization — Nokia 1100 HTML5 Port

While the core algorithm is designed in C++ for performance and complexity analysis, a web-based visualization was developed using HTML5 Canvas and JavaScript. This allows the game to be demonstrated in a browser environment while maintaining the same $O(n)$ logic used in the C++ version.

The HTML5 version faithfully recreates a 2005 Nokia 1100 handset using pure CSS — including the grey plastic shell, green-yellow LCD backlight (#c7d1af), clickable keypad, Nokia logo, speaker grille, and LCD scanline overlay. The game canvas renders on a 20×20 cell grid at 14px per cell.

Architecture Mapping — C++ to JavaScript

C++ Class / Concept	JS Equivalent	Notes
Point body[MAX_LEN]	Array of {x,y} objects	Same O(n) shift behaviour
Snake::move(grew)	snake.unshift() / pop()	Shift pattern preserved
Snake::hitSelf() O(n)	snake.some(s => s.x===...)	Linear scan — identical complexity
Wall collision	head.x < 0 head.x >= COLS	Game ends on any border hit — O(1)
Food::respawn()	spawnFood() with do-while	Avoids body overlap — O(n)
SnakeGame::logic()	tick() called by setInterval	Game loop via timer instead of sleep
Direction enum	String constants + DIR map	up/down/left/right with delta vectors
ScoreManager	score, hiScore variables	No file I/O in browser; kept in memory

Controls — Web Version

Input Method	Action
W / A / S / D keys	Move Up / Left / Down / Right
Arrow keys	Move in respective direction
Number keys 2, 4, 6, 8	Move Up / Left / Right / Down (Nokia keypad layout)
Number key 5 / Enter	Start game or restart after game over
Clickable keypad (2,4,6,8)	On-screen Nokia keys trigger direction changes
OK button / nav ring	Start or restart game
END key	Pause current game

Key Code Snippet — JS Wall Collision + Self-Collision (O(n))

```
// Wall collision: game ends when head leaves the grid - O(1)
const newHead = {
  x: head.x + DIR[dir].x,
  y: head.y + DIR[dir].y,
};
```

```
// Wall collision - O(1): game ends on any border hit
if (newHead.x < 0 || newHead.x >= COLS ||
    newHead.y < 0 || newHead.y >= ROWS) { endGame(); return; }

if (snake.some(s => s.x === newHead.x && s.y === newHead.y)) {
    endGame(); return;
}

// Dynamic body update - deque pattern (mirrors C++ move())
snake.unshift(newHead);          // push new head to front
if (!ate) snake.pop();           // remove tail unless growing
```

4. Core Algorithms

4.1 Grid Traversal Algorithm

The snake moves step-by-step in a 2D grid. Each game tick the snake's head position advances by one cell in the current direction. The grid (2D char array) is rebuilt every frame by iterating all rows and columns — $O(\text{WIDTH} \times \text{HEIGHT})$.

4.2 Dynamic Body Update Algorithm

This algorithm controls how the snake's body changes each tick. All body segments shift one slot to the right; the new head is placed at index 0. When growing, length is incremented first so the tail slot is preserved.

Scenario	Action
Food NOT eaten	Insert new head at front → remove tail (slide)
Food eaten	Insert new head at front only → no tail removal (grow)

4.3 Collision Detection Algorithm

- Wall Collision — $O(1)$: Check if $\text{head.x} \leq 0$, $\text{head.x} \geq \text{WIDTH}-1$, $\text{head.y} \leq 0$, or $\text{head.y} \geq \text{HEIGHT}-1$.
- Self Collision — $O(n)$: Linear scan of body segments from index 1 to $n-1$, checking if any segment equals the head position.

4.4 Pseudocode

```
SETUP:
snake ← [centre, centre-1, centre-2]
```

```

direction ← RIGHT,  score ← 0
place food at random unoccupied cell

GAME LOOP (O(n) per tick):
  read keyboard → update direction
  newHead ← currentHead + directionVector
  IF newHead == foodPosition:
    insert newHead at front    (snake grows)
    score += 10
    respawn food
  ELSE:
    insert newHead at front
    remove last element        (snake slides)
  IF newHead hits wall OR newHead in body[1..n]:
    gameOver ← true
  render grid,  sleep(SPEED_MS)

```

4 (Updated). Cross-Environment Empirical Analysis

4.2 Compiled vs Interpreted Environment Comparison

Experiment Setup: The $O(n)$ movement logic was tested in both a Compiled Environment (C++) and an Interpreted Environment (JavaScript running in a browser). The self-collision detection loop — which performs n comparisons per tick — was used as the benchmark operation.

Metric	C++ (Compiled)	JavaScript (Browser)
Execution model	Native machine code	JIT-compiled bytecode
Memory management	Direct, manual (stack/heap)	Garbage collected (GC pauses)
Small n (≤ 50)	Instant — imperceptible	Instant — imperceptible
Medium n (≤ 500)	Instant — imperceptible	Instant — imperceptible
Large n ($\leq 10,000$)	Handles smoothly	Minor GC pauses possible
Very large n ($> 10,000$)	Efficient — direct memory	Slower due to GC & boxing overhead
Nokia 20×20 grid (≤ 360)	Optimal	Equally optimal — no difference
Algo complexity	$O(n)$ — identical	$O(n)$ — identical

Observations

- C++ handles extremely large snake lengths ($n > 10,000$) more efficiently than JavaScript due to direct memory management and the absence of garbage collection pauses.

- For the Nokia-style 20x20 grid (maximum possible snake length = 360 cells), both C++ and JavaScript are equally efficient — the $O(n)$ scan completes in microseconds.
- The JavaScript `Array.prototype.some()` method implements the same linear $O(n)$ scan as the C++ for-loop, confirming algorithmic equivalence across environments.
- The main practical difference is memory overhead: C++ stores Point structs on the heap directly; JavaScript wraps each $\{x,y\}$ as a heap-allocated object with additional GC metadata.
- Conclusion: The algorithmic design is environment-agnostic. Porting from C++ to JS preserves all complexity properties for real-world game-scale inputs.

5. Complexity Analysis

5.1 Time Complexity

Operation	Time Complexity	Explanation
Snake movement (insert + pop)	$O(n)$	array shift moves $n-1$ elements right each tick
Wall collision check	$O(1)$	Checks only the head position
Self collision check	$O(n)$	Scans all $n-1$ body segments
Food spawn (worst case)	$O(n)$	Checks all body segments for overlap
Grid render	$O(W \times H)$	Iterates entire 40×20 grid each frame
Overall per game tick	$O(n)$	n = snake body length

5.2 Space Complexity

Structure	Space	Notes
Point body[] array (fixed-size)	$O(n)$	Grows with snake length
Grid buffer (2D char array)	$O(W \times H)$	Fixed $40 \times 20 = 800$ chars
Food & Score	$O(1)$	Single values only

6. Testing & Empirical Analysis

The following empirical analysis validates the theoretical $O(n)$ time complexity by observing actual behavior across different snake lengths. The input parameter used is the snake body length (n).

6.1 Test Results Table

Snake Length (n)	Comparisons Needed	Collision Time	Observed Behaviour
Small (n = 5)	~5 comparisons	Very Fast	Instant response
Medium (n = 20)	~20 comparisons	Fast / Moderate	No noticeable lag
Large (n = 50)	~50 comparisons	Slightly Slower	Minimal perceptible delay

6.2 Observations

- As snake length increases, the number of comparisons in self-collision detection increases proportionally.
- This confirms a linear $O(n)$ growth in processing time for the collision detection loop.
- Wall collision remains $O(1)$ regardless of snake length — only head position is checked.
- Food spawn may temporarily spike to $O(n)$ when the snake is very large, but this is rare.
- Overall game performance remains smooth for all tested lengths on modern hardware.

6.3 Test Cases

#	Test Case	Expected Result	Status
1	Press W → snake moves UP	Direction changes to UP	PASS
2	Snake reaches top wall	Game Over triggered	PASS
3	Snake head overlaps body segment	Self-collision → Game Over	PASS
4	Snake head reaches food position	Snake grows, score +10	PASS
5	Press S while moving UP	Direction unchanged (no reversal)	PASS
6	Press ENTER at welcome screen	Game starts	PASS
7	Press Q during game	Game exits cleanly	PASS
8	High score beaten	Saved to highscore.txt	PASS

6 (Updated). Real-World Application & Industry Relevance

6.1 C++ Engine + HTML/JS Frontend — Industry Practice

Using a high-performance compiled language (C++) for core logic while using HTML/JavaScript for the user interface is a well-established and widely-used architectural pattern in the software industry. This project directly demonstrates that principle at a small, educational scale.

Industry Example	C++ Layer	HTML/JS Layer
AAA Video Games (e.g. Unreal Engine titles)	Game engine, physics, AI, rendering	In-game menus, HUD overlays, settings screens (HTML-based UI via CEF)
Web Browsers (Chrome, Firefox)	V8/SpiderMonkey JS engine, rendering pipeline, networking	Browser UI: tabs, address bar, settings pages (HTML/CSS)
Desktop Apps (Spotify, Discord, VS Code)	Native C/C++ backend processes	Electron-based HTML/JS frontend
This Project (Snake Game)	C++ core: OOP classes, $O(n)$ collision, fixed-size array DS, memory management	HTML5 Nokia UI: CSS phone shell, Canvas game loop, JS port of same $O(n)$ logic

Why This Architecture Matters

- Separation of Concerns: The game logic (what happens) is independent of the presentation layer (how it looks). This makes both easier to test, modify, and extend.
- Performance where it counts: C++ handles computationally intensive tasks (physics, collision detection at scale, AI pathfinding) without garbage collection overhead.
- Reach and accessibility: HTML/JS frontends run on any device with a browser — no installation required. The Nokia HTML5 version of this Snake Game runs instantly on mobile, tablet, or desktop.
- Maintainability: UI designers can modify the HTML/CSS phone shell without touching any game logic. Game developers can optimise the C++ algorithms without touching the interface.
- This project demonstrates that a student can understand and apply this real-world pattern even at an introductory OOP level — validating both the academic and practical relevance of the work.

6.2 Innovation Summary

#	Innovation	Academic Value
1	Dual-platform implementation (C++ + HTML5/JS)	Demonstrates algorithmic portability and language-agnostic design
2	Nokia 1100 CSS skeuomorphic UI	Shows creative application of web technologies beyond typical usage
3	Wall collision detection (realistic game-over)	Matches the classic Snake behaviour — game ends on border hit, not wraparound
4	Cross-environment complexity comparison	Extends empirical analysis beyond a single language/runtime

5	Clickable Nokia keypad controls	Event-driven input mapping — mirrors embedded systems programming
---	---------------------------------	---

7. Team Contributions & Time Distribution

7.1 Team Members & Roles

Member	Role	Responsibilities
Ibrar Ahmed	Implementation	Complete C++ code, game logic, movement system, collision detection, scoring system, high score persistence
M. Sanan	Documentation	Project report, pseudocode, algorithm explanations, class diagrams, and written analysis
Hanzala Baqir	Testing & Analysis	Designed test cases, conducted empirical testing, observed complexity behavior, validated game features

7.2 Time Distribution

Task	Time Taken
Design & Planning	3 hours
Coding & Implementation	1 day
Testing	3 hours
Documentation	2 hours

8. GitHub

<https://github.com/HanzallaBaqir/Snake-Game-/upload>

9. Conclusion

This project successfully demonstrates how fundamental computer science concepts — Object-Oriented Programming, Data Structures, and Programming Fundamentals — can be applied to build a complete, working software application.

The Snake Game was implemented in two forms: a C++ console application using five OOP classes and a fixed-size Point array with shift-based movement, and an HTML5/JavaScript Nokia 1100-themed browser version that ports the identical $O(n)$ algorithm to a web canvas. Both versions share the same algorithmic core — confirming that good software design is language-agnostic.

The cross-environment empirical analysis showed that for Nokia-scale grids ($20 \times 20 = 360$ max cells), C++ and JavaScript perform identically. C++ holds a meaningful advantage only at extreme snake lengths ($n > 10,000$) due to direct memory management. This validates the practical relevance of the $O(n)$ complexity analysis in real-world contexts.

The architecture — C++ logic layer + HTML/JS presentation layer — mirrors the design used in professional AAA games, desktop applications, and modern browsers, giving this academic project direct relevance to industry practice.

Key Takeaways

- OOP makes complex systems manageable by dividing them into focused, reusable classes.
- Choosing the right data structure (fixed-size array with shift movement) keeps the implementation simple and efficient.
- The same $O(n)$ algorithm works identically in C++ and JavaScript — algorithms are language-agnostic.
- C++/native + HTML/JS is a real industry pattern used in games, browsers, and desktop apps.
- Cross-platform code requires platform-specific handling (conio.h vs termios / browser events).
- Empirical testing across environments validates theoretical complexity predictions.

— End of Report —